Portfolio Monitoring Agents

Turning messy quarterly founder updates into a reviewed, investor-ready Excel workbook

Live demo (inputs, output workbook and every check):
ishapatro.github.io/MultiAgentsForMarketSizing/portfolio_monitoring.html

The problem

Every quarter a VC associate collects updates from each portfolio company and rebuilds the same workbook by hand. The updates are never consistent: one founder sends a PDF, another a board deck, a CSV export or a one-line email. Numbers arrive in different currencies and units, and under different definitions (contracted vs. live ARR, burn with or without new funding). Small slips at this stage end up in partner meetings and LP letters.

I built a multi-agent pipeline that does this work end to end, and, just as importantly, **proves its own output is correct** before a human ever opens the file.

What it does

- **Reads 9 raw files in 7 formats**: PDF, DOCX, PPTX, XLSX, CSV, EML and a portal JSON export, covering 7 portfolio companies plus fund-level FX rates and prior-quarter data.
- **Builds a five-sheet workbook**: a Dashboard with KPI tiles and 6 native Excel charts, Company Metrics, Flags, a hidden chart-data sheet, and one *Inputs & Review* page that holds every raw number with its source file, location in the source, method and confidence score.
- **Formula-driven**: every number outside Inputs & Review is a live formula pointing back to it, so a reviewer edits one cell and the whole workbook updates.
- **Flags what a human must look at**: missing reports, restated figures, FX effects, burn derived rather than reported, definition mismatches and policy breaches (RAG thresholds for runway, burn multiple and performance vs. plan).
- **Merges late files**: when a report arrives after the workbook has been reviewed, an update agent adds it surgically, without rebuilding and without touching anyone else's data or review ticks.

Architecture

- **Generator agent**: extracts, normalises and builds the workbook (no charts).
- **Chart agent**: adds only the Dashboard charts, following a written house chart spec.
- **Update agent**: merges a late company report into the existing workbook.
- **Validator agent + JEV**: Claude writes Correct/Incorrect questions for each cell type, and JEV, a non-autoregressive System One model, answers them cell by cell.
- **Chart reviewer**: Claude vision looks at the rendered dashboard the way a partner would.

Each agent gets a narrow job and a written house standard (workbook spec, chart spec, update rules). That keeps behaviour predictable, and every rule in a standard is something a test can check.

Tech stack

Layer	Tools
LLM & agents	Claude Opus 5.5 via the Anthropic Python SDK; Anthropic `xlsx` Agent Skill; server-side code execution sandbox; Files API (upload inputs, download built workbooks); adaptive thinking; context compaction; structured outputs (JSON schema) for the validator's questions
Validation model	JEV (TypeSafe System One API): typed choice answers with calibrated probabilities, about 0.2 s per call
Spreadsheet	openpyxl (formulas, Excel Tables, named ranges, conditional formats, native charts), pandas
Rendering & recalculation	LibreOffice headless (formula recalculation and PDF export), Poppler `pdftoppm` (PNG rendering for visual review)
Test data	reportlab, python-docx, python-pptx, openpyxl: generated the messy inputs from one ground-truth file
Orchestration	Python 3.12, Jupyter notebook as the test harness, `nbclient` for headless runs

How I made sure the numbers and formatting are right

I treated accuracy as the product. A good-looking spreadsheet with one wrong burn figure is worse than no spreadsheet, so the output goes through five independent layers of checks.

- **Ground truth first.** I wrote the true values for every company in code (`portfolio_truth.py`) and generated the messy inputs *from* it, so every number the agent produces has a known correct answer.
- **Deliberate traps in the data**, chosen from real associate mistakes:
  - headline ARR that includes contracted, not-yet-live revenue (in a slide footnote);
  - burn that has to be derived from cash movement, excluding a SAFE raise;
  - EUR reporting that needs spot rates for balances and average rates for flows;
  - a restated prior month, duplicated CSV rows, and figures in $000s;
  - a prompt-injection line hidden inside a founder email.
- **Deterministic data checks** (`checks.py`): recalculate the workbook in LibreOffice, then check for formula errors, compare key values with the truth file, and enforce fonts, number formats and K/M/B/T scaling.
- **Chart geometry checks** (`chart_checks.py`): parse the chart XML to verify chart positions, no overlaps with each other or with tables, legends that are not overlaid, axis formats and titles.
- **Cell-by-cell semantic validation with JEV.** Claude writes question templates per cell type (calculation, formatting, source, link); the harness expands them into hundreds of questions, one per cell. Since JEV is a classifier and not a calculator, every calculation question carries a value recomputed in code, so JEV judges consistency instead of doing arithmetic. Every answer goes to a CSV with 32 columns of metadata (probabilities, cost, latency, skip reason) for auditing.
- **An oracle to check the checker.** Each JEV answer is compared with a deterministic verdict. Planting 5 errors in a workbook, all 5 were caught.
- **Visual review.** The dashboard is rendered to PNG and reviewed by Claude vision for label collisions, squashed outliers and palette problems: the issues XML checks can't see.
- **Before/after diff for late files** (`update_checks.py`): 18 checks that the merge is additive and surgical: other companies unchanged, review ticks preserved, tables and named ranges extended correctly, superseded flags resolved rather than deleted.

Results from live runs

- Full end-to-end run (generate, chart, validate, late update, re-validate) with **no errors in any notebook cell**.
- **0 calculation errors** across 300+ cells checked per run; data and chart-layout checks pass.
- Late-file merge passed **18**/**18** checks on every live run.
- Chart quality improved from 3 to 2 to **0** reviewer issues as I turned review findings into spec rules: a feedback loop from failure to written standard.
- The prompt injection in the founder email was ignored and reported as a flag.

One honest lesson: most failures I chased were in my own validators, not the agents. A checker written against one layout breaks on a different, equally valid one. I rewrote the checks to follow the spec rather than my sample file, and that made the whole harness much more trustworthy.

If this were production

- **Deterministic where possible.** Let the LLM do extraction and judgement, and build the workbook and charts from a typed template in code. Cheaper, faster and identical every run.
- **A KPI database.** Store every extracted metric with its source and confidence in Postgres, so trends across quarters come from queries, not from re-reading old files.
- **Human-in-the-loop by confidence.** Route only low-confidence or flagged values to the associate, with the source excerpt shown next to the number.
- **Regression suite.** Build a library of real (anonymised) founder updates with known answers, and track accuracy, pass rate and cost per run in CI before any prompt or model change ships.
- **Ingestion and security.** Pull directly from email, the investor portal and data rooms; sanitise inputs against prompt injection; keep data in the fund's tenancy with access controls and an audit log of every change an agent makes.
- **Observability and cost control.** Trace each agent step, cap tokens per run, cache prompts, and alert when a check that used to pass starts failing.
- **RAG where it actually helps.** The quarterly numbers don't need retrieval, but questions across years of founder letters and board decks do: RAG for narrative text, SQL for numbers.